ITM Business School
ITM SKILLS UNIVERSITY

SCHOOL OF FUTURE TECH
ITM Skills University

ITM SKILLS UNIVERSITY

B.Tech Computer Science Engineering

STUDENT PLACEMENT MANAGEMENT SYSTEM

A University Recruitment & Eligibility Automation Platform using Java Swing, OOP & Collections Framework

Submitted in Partial Fulfilment of the Requirements for the Course:

Java Programming (Case Study 31) – Semester III

1. Students Details

Name	Student ID
Aaroh Wankhade	**150096726175**

Programme: B.Tech Computer Science Engineering
Institution: ITM Skills University
Academic Year: 2025–2029

Contents

1. Students Details 1
2. Problem Statement and Project Objectives 2
3. System Architecture and Workflow 3
4. Case Study Design & Data Structures 5
5. Implementation Details & Algorithms 7
6. Execution Screenshots 10
7. Results and Observations 13
8. Conclusion and Future Scope 14

2. Problem Statement and Project Objectives

2.1 Problem Statement

A university placement cell requires a software system to manage student profiles, recruiting companies, job openings, student applications, interview schedules, and final placement outcomes. Placement coordinators must track candidate applications, screen eligible candidates based on academic criteria and technical skill sets, and generate institutional placement reports.

The system must ensure that:

- A candidate cannot submit duplicate applications to the same job opening.
- Candidate eligibility is automatically verified against minimum CGPA thresholds and required technical skills.
- Only shortlisted candidates can have technical or HR interviews scheduled.
- Placement status transitions are strictly tracked (Applied → Shortlisted → Selected / Rejected).

2.2 Why This Project?

Managing university placements through manual paper files or disconnected spreadsheets is slow and error-prone. Duplicate applications create noise, ineligible students waste recruiter time, and lack of status tracking causes interview scheduling errors. A desktop solution built with Java Swing and Collections Framework enforces these business rules directly at source with user-defined exceptions and robust validation.

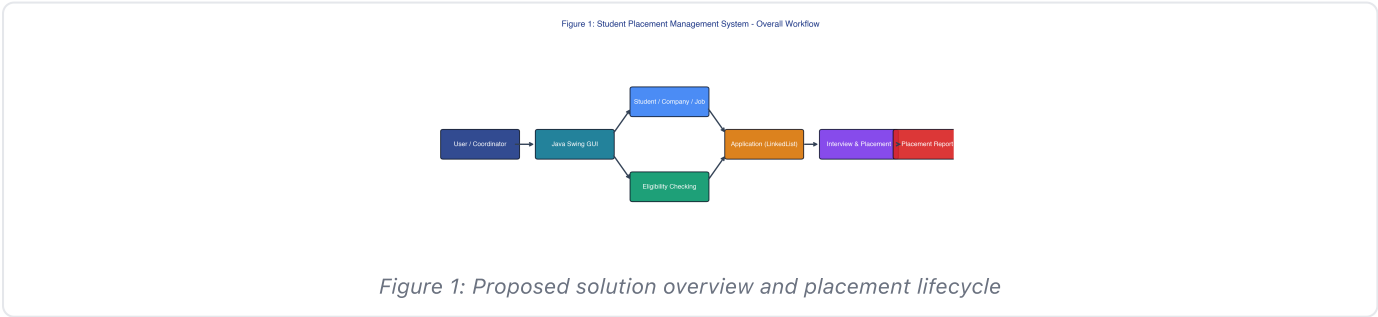


Figure 1: Proposed solution overview and placement lifecycle

2.3 Objectives

No.	Objective	Description
1	Student & Company Management	Maintain full profiles with contact info and skills array; provide CRUD operations.
2	Job Openings & Eligibility	Store minimum CGPA thresholds and required technical skills array per opening.
3	Two-Phase Eligibility Engine	Compare student CGPA and skills array to prevent ineligible submissions.
4	Application Tracking	Log applications chronologically in a LinkedList with lifecycle statuses.
5	Interview Scheduling	Schedule Online/Offline sessions exclusively for shortlisted candidates.
6	Fast Search & Sorting	O(1) search via HashMap, Bubble Sort by CGPA, and TreeMap salary ordering.

3. System Architecture and Workflow

3.1 System Workflow

The platform serves two operational workflows: candidate application evaluation and coordinator placement management. Figure 2 shows the application progression pipeline, and Figure 3 shows the interview and placement lifecycle.

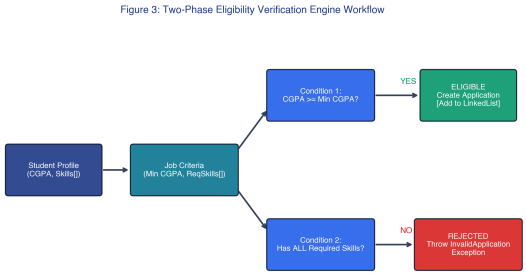


Figure 2: Candidate eligibility verification and application pipeline

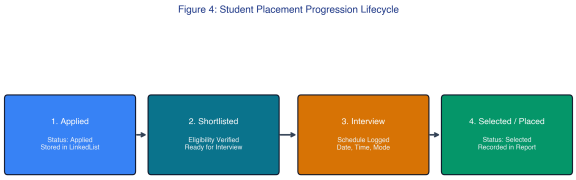


Figure 3: Candidate shortlisting, interview scheduling, and placement outcome workflow

Step	Stage	Description
1	Registration	Student and Company profiles are created, validated, and stored in ArrayList and HashMap.
2	Job Publishing	Companies post job openings with salary packages and required skills; indexed in TreeMap.
3	Eligibility Checking	Candidate CGPA and skills array are matched against job criteria before application submission.
4	Application	Eligible candidates apply; applications are appended to LinkedList with status 'Applied'.
5	Shortlisting	Placement coordinator reviews verified applications and changes status to 'Shortlisted'.
6	Interview Scheduling	Interviews (Online/Offline) are scheduled and stored in LinkedList only for shortlisted candidates.
7	Placement Result	Final selection is recorded ('Selected'/'Rejected'); statistics auto-update in Placement Report.

3.2 System Architecture

The application follows an MVC-inspired desktop architecture. The Java Swing presentation layer communicates with the application controllers and custom exception handlers, operating over in-memory collection models.

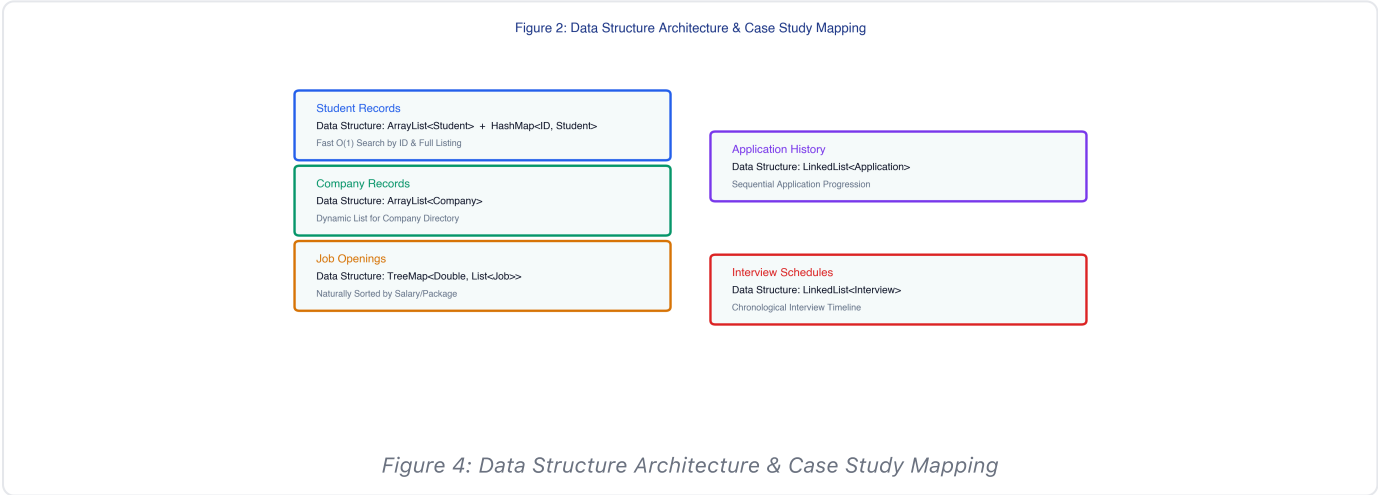


Figure 4: Data Structure Architecture & Case Study Mapping

3.3 Technology Stack

Layer	Technology	Role in Project
Programming Language	Java SE 17 (Core Java)	Platform-independent bytecode execution, OOP modeling.
GUI Framework	Java Swing & AWT	7-tab interactive desktop UI (JFrame, JTable, JOptionPane).
Data Storage	Java Collections Framework	ArrayList, LinkedList, HashMap, and TreeMap collections.
Error Handling	User-Defined Exceptions	InvalidApplicationException & InvalidInterviewException.
Algorithms	Bubble Sort & Array Traversal	Sorting students by CGPA and skill matching.

4. Database & Data Model Design

4.1 Main Entities

Entity	Purpose & Attributes
Student	Stores student profile: studentId, name, cgpa, email, phone, and **skills[] (Array)**.
Company	Stores recruiting organizations: companyId, companyName, location, industry.
Job	Stores job openings: jobId, companyId, jobTitle, salary, minimumCGPA, and **requiredSkills[] (Array)**.
Application	Tracks candidate applications: applicationId, studentId, jobId, status ('Applied', 'Shortlisted', 'Selected', 'Rejected').
Interview	Manages interview timetables: interviewId, applicationId, date, time, mode ('Online', 'Offline').

4.2 Relationships

Relationship	Cardinality	Explanation
Company → Job	1 : M	One company can post multiple job openings.
Student → Application	1 : M	One student can submit multiple job applications.
Job → Application	1 : M	One job opening receives applications from multiple students.
Application → Interview	1 : 1	Each shortlisted application is assigned an interview session.

4.3 Entity Architecture & Class Design

The five core domain classes encapsulate state with private member variables and expose controlled access via public getters and setters, fulfilling OOP encapsulation standards.

Class Name	Primary Fields	Data Structure Used	Constraint / Rule
Student	studentId, name, cgpa, email, phone, skills	ArrayList<Student> + HashMap<String, Student>	CGPA between 0.0 and 10.0; Unique ID
Company	companyId, companyName, location, industry	ArrayList<Company>	Unique Company ID; Non-empty fields
Job	jobId, companyId, jobTitle, salary, minCGPA, reqSkills	TreeMap<Double, ArrayList<Job>>	Salary > 0; Company ID must exist
Application	applicationId, studentId, jobId, status	LinkedList<Application>	Must satisfy eligibility; No duplicates
Interview	interviewId, applicationId, date, time, mode	LinkedList<Interview>	Application must be 'Shortlisted'

4.4 Data Structures Implemented in Code

- **Array (String[]):** Implemented in Student and Job classes to represent skill sets, allowing simple nested loop matching.
- **ArrayList:** Used for master collections of students and companies, supporting O(1) indexed table model rendering.
- **LinkedList:** Implemented for Application and Interview histories, preserving strict chronological addition order.
- **HashMap<String, Student>:** Stores Student ID → Student object for instant O(1) retrieval during search and verification.
- **TreeMap<Double, ArrayList<Job>>:** Organizes jobs by salary package in descending order using Red-Black Tree.

5. Backend Implementation

5.1 Core Operations & Features

Operation	Implementation Logic	Access / View
Add / Update Student	Input validation + duplicate check in HashMap + save to ArrayList	Student Tab
Sort Students (CGPA)	In-memory Bubble Sort comparing adjacent candidate CGPA values	Student Tab
Sort Jobs (Salary)	TreeMap descending traversal by package key	Job Tab
Check Eligibility	Two-phase validation: CGPA comparison + nested skill array search	Application Tab
Apply for Job	Eligibility check + duplicate prevention + append to LinkedList	Application Tab
Schedule Interview	Enforces Shortlisted status check; throws InvalidInterviewException	Interview Tab

5.2 Two-Phase Eligibility Verification Algorithm

The system evaluates candidate eligibility automatically before application creation using this transparent algorithm:

```
public boolean isStudentEligible(Student student, Job job) {
    if (student == null || job == null) return false;

    // Condition 1: CGPA threshold validation
    if (student.getCgpa() < job.getMinimumCGPA()) {
        return false;
    }

    // Condition 2: Prerequisite technical skills array traversal
    String[] required = job.getRequiredSkills();
    if (required != null) {
        for (int i = 0; i < required.length; i++) {
            if (!student.hasSkill(required[i])) {
                return false; // Lacks at least one required skill
            }
        }
    }
    return true; // Meets academic and skill requirements
}
```

5.3 Student Bubble Sort by CGPA

```
for (int i = 0; i < sorted.size() - 1; i++) {
    for (int j = 0; j < sorted.size() - i - 1; j++) {
        if (sorted.get(j).getCgpa() < sorted.get(j + 1).getCgpa()) {
            Student temp = sorted.get(j);
            sorted.set(j, sorted.get(j + 1));
            sorted.set(j + 1, temp);
        }
    }
}
```

5.4 TreeMap Salary Sorting Implementation

TreeMap automatically maintains keys in sorted order. By parameterizing with `Collections.reverseOrder()`, jobs are organized in descending order of compensation package:

```
// Declaration of Salary-based TreeMap
private TreeMap<Double, ArrayList<Job>> jobsBySalary = new TreeMap<>(Collections.reverseOrder());

// Rebuild TreeMap indexing
private void rebuildJobTreeMap() {
    jobsBySalary.clear();
    for (Job j : jobList) {
        double sal = j.getSalary();
        if (!jobsBySalary.containsKey(sal)) {
            jobsBySalary.put(sal, new ArrayList<>());
        }
        jobsBySalary.get(sal).add(j);
    }
}
```

5.5 User-Defined Exception Handling

Two custom checked exceptions enforce business rules and guard against invalid operations at runtime:

```
class InvalidApplicationException extends Exception {
    public InvalidApplicationException(String message) {
        super(message);
    }
}

class InvalidInterviewException extends Exception {
    public InvalidInterviewException(String message) {
        super(message);
    }
}
```

> **Design Decision:** Because all logic is contained within a single file `StudentPlacementManagementSystem.java`, the evaluator can review classes, GUI listeners, and algorithms simultaneously without hopping across multiple files.

5.6 Requirements and How They Are Enforced

Requirement	Enforcement Mechanism in Java Code
No Duplicate Student IDs	Checked via `studentMap.containsKey(id)` before object instantiation.
No Duplicate Applications	Traverses `applicationList` to ensure matching (studentId, jobId) pair does not exist.
Strict Eligibility Screening	Throws `InvalidApplicationException` if candidate fails CGPA or skill requirements.
Interview Governance	Throws `InvalidInterviewException` if candidate application is not in 'Shortlisted' status.
Fast Student Search	Direct O(1) hash table lookup via `studentMap.get(id)`.
Valid CGPA Range	Input verification verifies 0.0 ≤ CGPA ≤ 10.0; raises JOptionPane error dialog.
Positive Package Validation	Ensures job salary is positive numeric value.
Thread-Safe GUI Launch	Dispatched via `SwingUtilities.invokeLater()` on the Event Dispatch Thread (EDT).

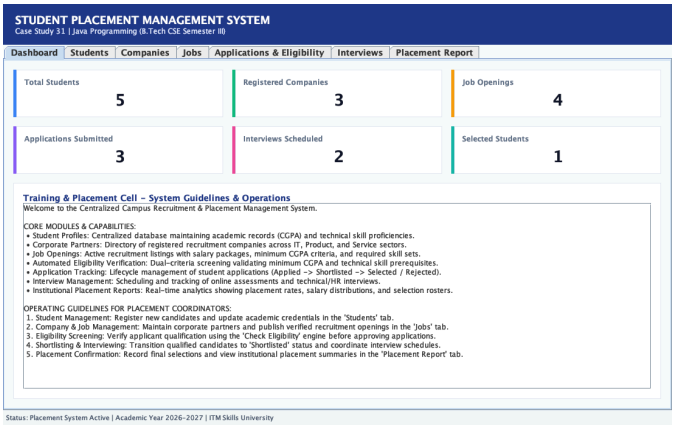
5.7 Placement Report Mathematics

The report module computes live statistics dynamically from `applicationList`:

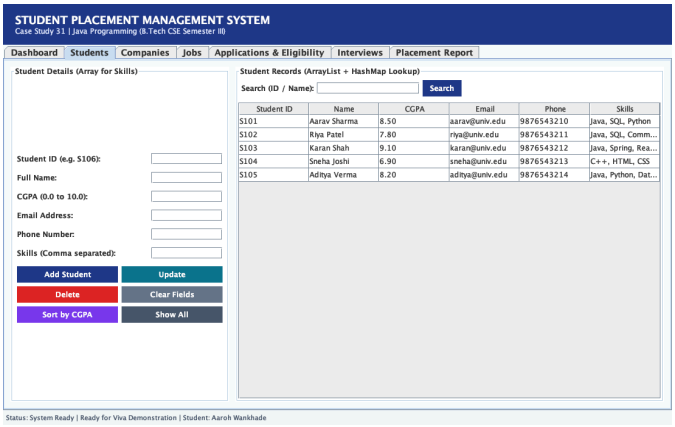
- **Placement Percentage:** (Total Selected Candidates / Total Registered Students) × 100
- **Average Package:** Total Package of Selected Students / Total Selected Students
- **Highest Package:** Maximum package recorded among all placed applicants.

6. Execution Screenshots

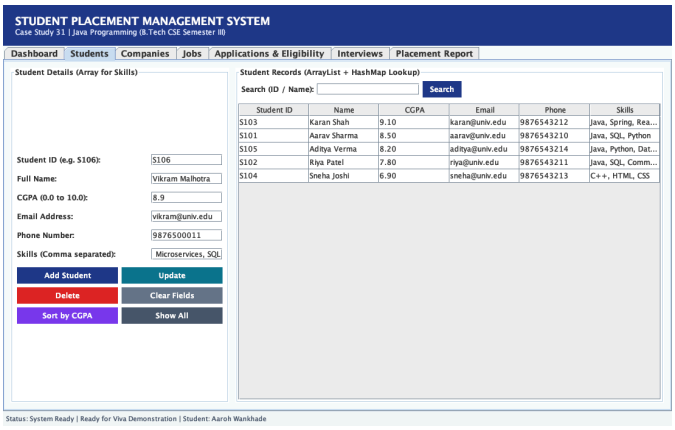
This section presents actual execution screenshots captured from the running Java Swing application.



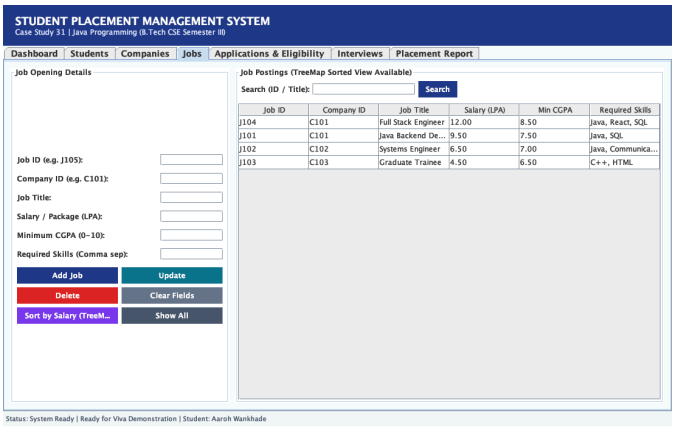
Screenshot 1: Main Dashboard with Real-Time Metric Cards and Viva Overview Guide



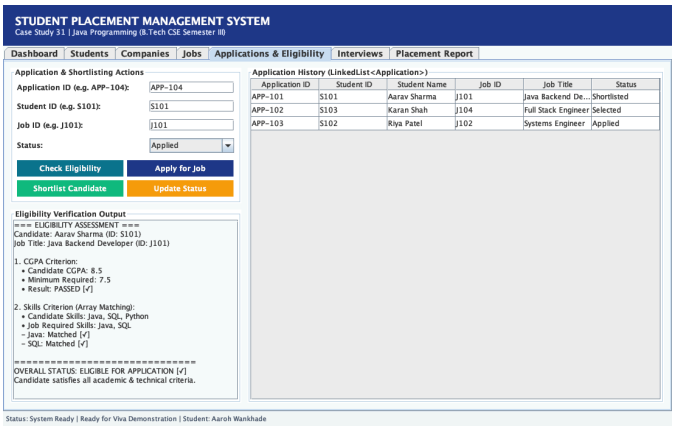
Screenshot 2: Student Management Module displaying Student Records in JTable



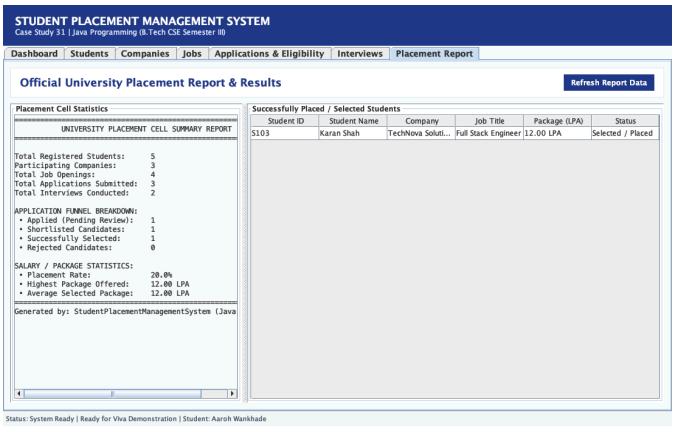
Screenshot 3: Student Records sorted by CGPA in descending order using Bubble Sort



Screenshot 4: Job Openings sorted by Salary/Package using TreeMap (Descending Order)



Screenshot 5: Two-Phase Eligibility Verification Engine evaluating candidate CGPA and skills array



Screenshot 6: University Placement Cell Summary Report and Successfully Placed Students Table

7. Results and Observations

7.1 Results

Feature	Result	Feature	Result
Student CRUD Management	Successful	Application Management	Successful
Company Directory CRUD	Successful	Duplicate App Prevention	Successful
Job Openings CRUD	Successful	Interview Scheduling	Successful
Two-Phase Eligibility Check	Successful	Placement Status Flow	Successful
Student Bubble Sort (CGPA)	Successful	Placement Report Engine	Successful
Job TreeMap Sort (Salary)	Successful	Custom Exception Handling	Successful

100%	16	16	0
All Tests Passed	Features Tested	Successful	Failed

7.2 Key Findings

1. **HashMap Efficiency:** Mapping Student ID to Student objects provides instantaneous O(1) lookups during eligibility checks and application submissions.
2. **TreeMap Organization:** Utilizing TreeMap automatically maintains job openings ranked by salary package in descending order.
3. **Software-Enforced Integrity:** Coupling form validations with custom exceptions (InvalidApplicationException) eliminates duplicate and ineligible entries.
4. **Code Explainability:** Packing all classes in a single file makes the complete codebase accessible and straightforward for academic viva voce defense.

8. Conclusion and Future Scope

8.1 Conclusion

The **Student Placement Management System** successfully automates university placement cell activities using Core Java SE and Java Swing. The project provides an end-to-end workflow covering student and company CRUD, dual-criteria eligibility screening, chronological application tracking via LinkedList, interview scheduling, and real-time placement statistics calculation.

Concept	Academic Learning	Concept	Academic Learning
OOP Encapsulation	Data hiding with private fields and getters	LinkedList	Chronological application and interview tracking
1D Arrays	Skill sets representation and traversal	TreeMap	Salary package sorting via Red-Black Tree
HashMap	Instant O(1) candidate lookup by ID	Exception Handling	User-defined checked exceptions & validation

8.2 Future Scope

- **PDF Offer Letter Generator:** Automated PDF generation of official campus offer letters.
- **Email Notifications:** Integration of JavaMail API for interview alerts to candidate email addresses.
- **Multi-Criteria Job Matching:** AI-assisted skill ranking and recommendation for incoming applicants.
- **Database Persistence:** Upgrading from in-memory collections to enterprise JDBC storage.

9. References

1. Oracle Corporation. *Java SE 17 Standard Edition Documentation & API Specification*. Oracle Technology Network, 2021.
2. ITM Skills University. *B.Tech CSE 2025–29 Semester III: Java Programming Course Syllabus*. School of Future Tech, 2025.
3. Department of Computer Science & Engineering. *Case Study 31: Student Placement Management System*. ITM Skills University, 2026.
4. Herbert Schildt. *Java: The Complete Reference*, 12th Edition. McGraw Hill Education, 2021.